

RELAZIONE HOMEWORK 3 DSBD

Luca Eterno – Rosario Grasso

La presente relazione si pone in continuità con le versioni precedenti e si limita a descrivere le modifiche e le integrazioni introdotte, aggiornando lo stato complessivo dell'applicazione. Si suggerisce pertanto una lettura congiunta dei tre documenti al fine di cogliere pienamente il contesto e l'evoluzione del progetto

I. Abstract

Il progetto realizza un sistema distribuito per la gestione delle utenze, il monitoraggio di aeroporti di interesse e la raccolta automatizzata dei dati di volo tramite le API di OpenSky Network.

L'architettura è composta da quattro microservizi principali:

- **User Manager:** Gestisce le registrazioni degli utenti
- **Data Collector:** Gestisce gli aeroporti di interesse e il recupero dai dati di volo ed è protetto da un **Circuit Breaker** per la resilienza verso le API esterne.
- **Alert System:** Analizza gli eventi di volo rispetto alle soglie definite dagli utenti.
- **Alert Notifier System:** Gestisce l'invio automatizzato di notifiche via email (integrazione MailHog).

I microservizi si coordinano attraverso vari modelli di comunicazione:

- **REST:** utilizzato per tre flussi critici: l'interazione tra il client e il sistema, il monitoraggio con Prometheus e l'integrazione con OpenSky Network.
- **gRPC:** Utilizzata per la verifica delle credenziali.
- **Kafka:** Spina dorsale del sistema per la propagazione degli eventi tra Data Collector, Alert System e Alert Notifier System.

Vengono utilizzate due tipologie di database:

- **Redis:** Implementazione della politica *at-most-once*.
- **MySQL:** Due database relazionali distinti per garantire registrazioni degli utenti e dati dei voli.

L'evoluzione finale del progetto vede il passaggio da Docker Compose a Kubernetes e l'integrazione di Prometheus:

- **Kubernetes (KIND):** L'intero sistema è distribuito su un cluster K8s, gestendo il routing del traffico tramite ingress controller, la persistenza tramite i **Persistent Volume Claims (PVC)** e le configurazioni tramite ConfigMaps e Secrets.
- **Prometheus:** I microservizi principali (user manager e data collector) vengono monitorati attraverso un sistema di whitebox monitoring dove prometheus esegue lo scraping delle metriche esposte su uno specifico endpoint

II. Nuove funzionalità

Prometheus

L'integrazione di Prometheus nell'HW3 permette di monitorare in tempo reale le prestazioni dei microservizi User Manager e Data Collector. Il sistema si basa sull'uso delle label che garantiscono la dimensionalità dei dati: mentre la label `service` identifica staticamente il servizio di origine della metrica, la label `node` cattura la dinamicità di Kubernetes, tracciando su quale nodo del cluster è in esecuzione il Pod per isolare eventuali problemi infrastrutturali.

In entrambi i microservizi sono state implementate le metriche counter: `http_requests_total` ed `errors_total`. La prima permette di tracciare il volume di traffico, la seconda gli errori interni del sistema. Grazie alle label `endpoint` e `method` si possono filtrare le metriche per l'endpoint che le ha generate, così da poter individuare quella con maggior traffico o quella che genera errori.

Accanto alle metriche counter sono state implementate le metriche gauge: `response_time` e `db_update_time` che misurano rispettivamente la latenza delle risposte e la durata delle query MySQL. La prima adotta le stesse label delle metriche counter mentre la seconda utilizza una label più specifica di `endpoint` ossia `operation` per individuare l'esatta operazione che si sta compiendo sul database.

Infine, a Data Collector è stata aggiunta la metrica gauge `opensky_fetch_time` per tenere traccia dei tempi di risposta delle API esterne. Al fine di ottenere informazioni complete per tutti gli interessi registrati dagli utenti, la metrica è stata implementata in maniera tale da distinguere i tempi di fetch per ogni singolo aeroporto. Avendo un numero limitato di aeroporti questo non causerà un numero eccessivo di valori ma permetterà di individuare eventuali problemi con determinati aeroporti da parte di Opensky.

Kubernetes

La distribuzione dell'applicazione è stata realizzata tramite `kind` per eseguire cluster Kubernetes (K8s) in locale utilizzando container Docker come nodi. La configurazione del cluster prevede due nodi: un control plane e un worker. Il primo si occupa della gestione del cluster mentre il secondo della mera esecuzione dei POD. In questo modo si dividono le responsabilità e il sistema può scalare in maniera orizzontale aggiungendo worker.

L'accesso dall'esterno è regolato da un NGINX Ingress Controller, che agisce come gateway centralizzato applicando le regole di routing verso i microservizi principali, ovvero User Manager e Data Collector. Al contrario, gli strumenti di analisi e supporto come Prometheus e MailHog sono esposti tramite servizi di tipo NodePort, garantendo un accesso diretto per finalità di monitoraggio e debug. Per assicurare la persistenza dei dati, i database, Kafka e Prometheus si appoggiano a Persistent Volume Claims (PVC), in modo da separare i dati dal ciclo di vita effimero dei container.

Infine, mentre i database e Prometheus vengono inizializzati tramite ConfigMap per il caricamento di parametri statici, Kafka richiede l'esecuzione di un Job dedicato per la creazione dei topic una volta che il broker è attivo. La sicurezza dell'intero ecosistema è garantita dall'uso dei Secret, che proteggono password e credenziali sensibili evitandone l'esposizione in chiaro nei file di configurazione.

III. Schema Architettrale

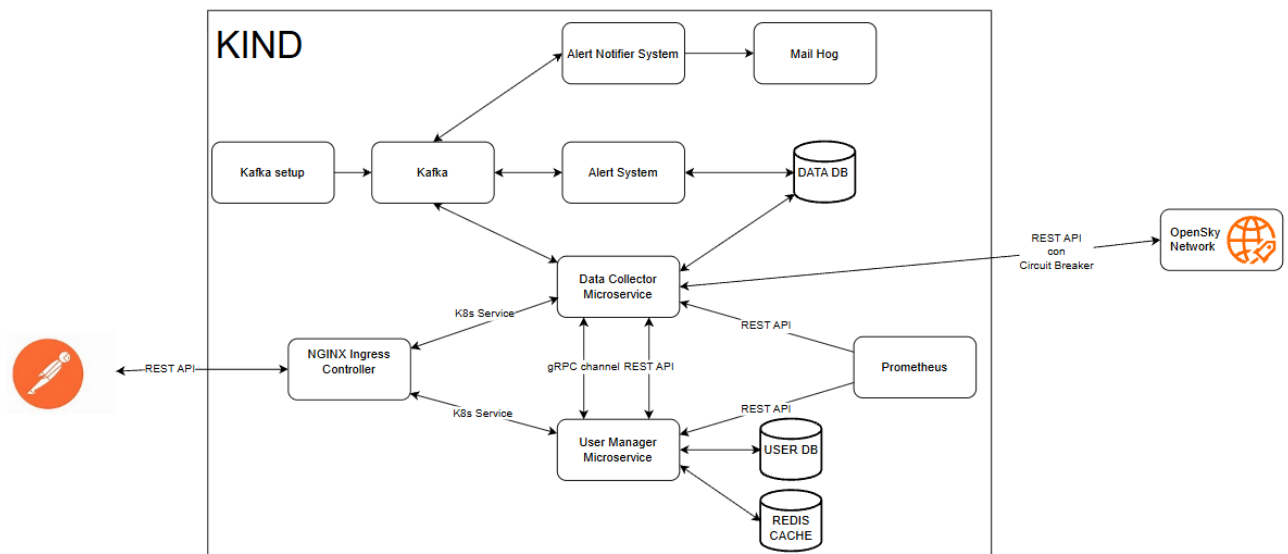


Figura 1: Schema Architettrale

Il diagramma architettrale mostrato in figura 1 aggiunge rispetto al homework 2:

- **KIND:** L'intero sistema è racchiuso in un cluster Kubernetes. Questo passaggio ha trasformato i microservizi in POD gestiti dal Control Plane.
- **NGINX Ingress Controller:** L'API Gateway precedentemente introdotto è stato sostituito da un Ingress Controller nativo di Kubernetes. Esso funge da punto di ingresso unico per il traffico REST esterno, gestendo il routing verso i microservizi Data Collector e User Manager.
- **Prometheus:** Prometheus interagisce tramite API REST con i microservizi per effettuare lo scraping delle metriche.
- **Kafka Setup:** Un job Kubernetes che si occupa della creazione dei topic necessari.

IV. UML

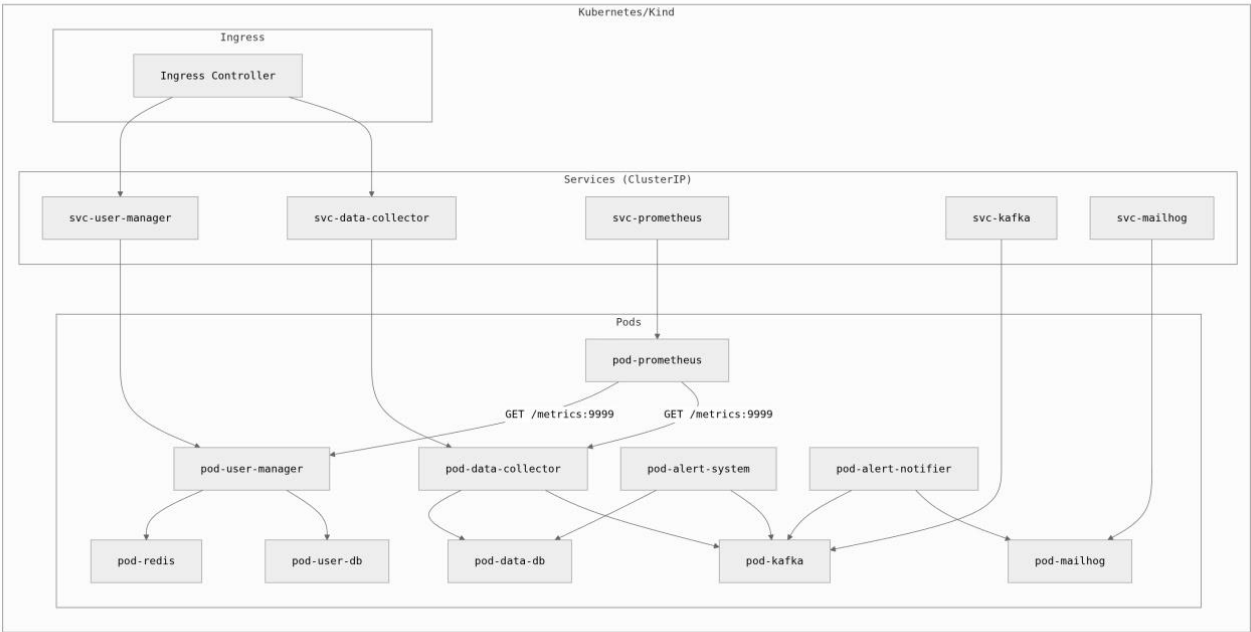


Figura 2: Diagramma architetturale

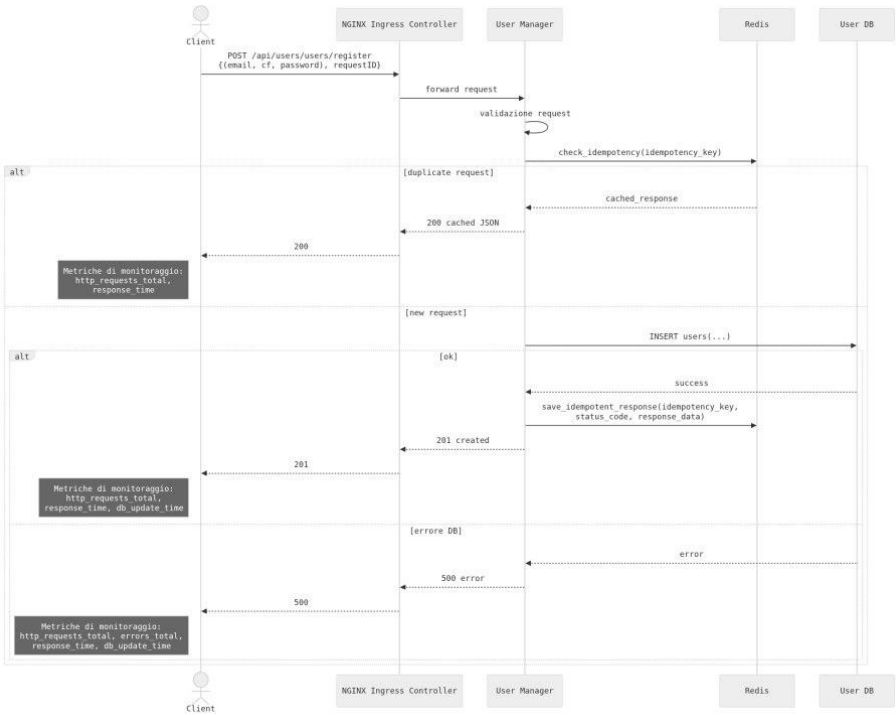


Figura 3: UML - User Manager - Registrazione utente

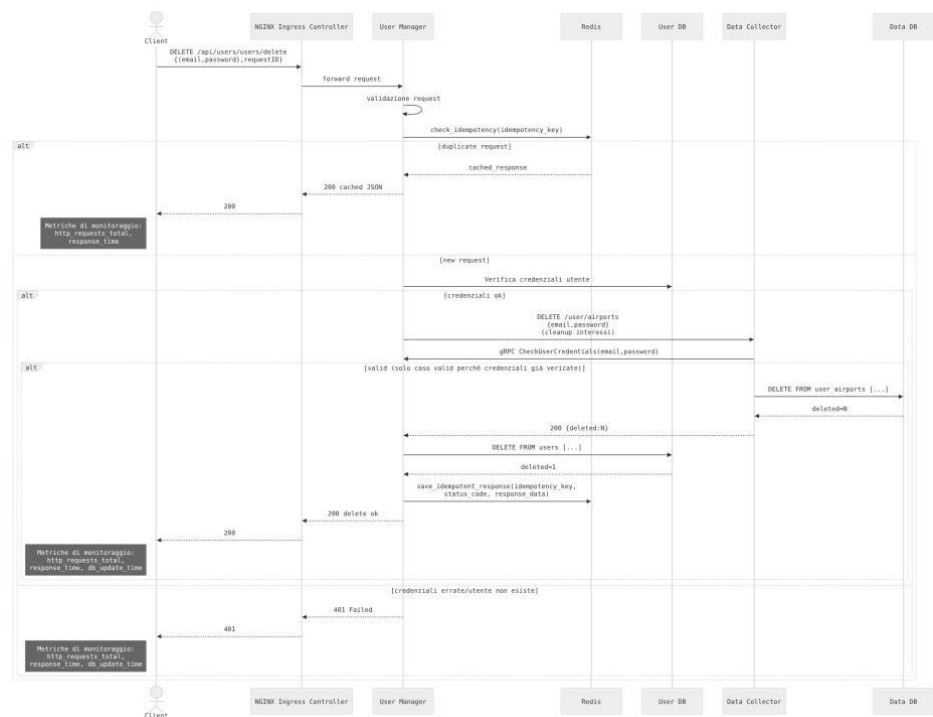


Figura 4: UML - User Manager - Cancellazione utente

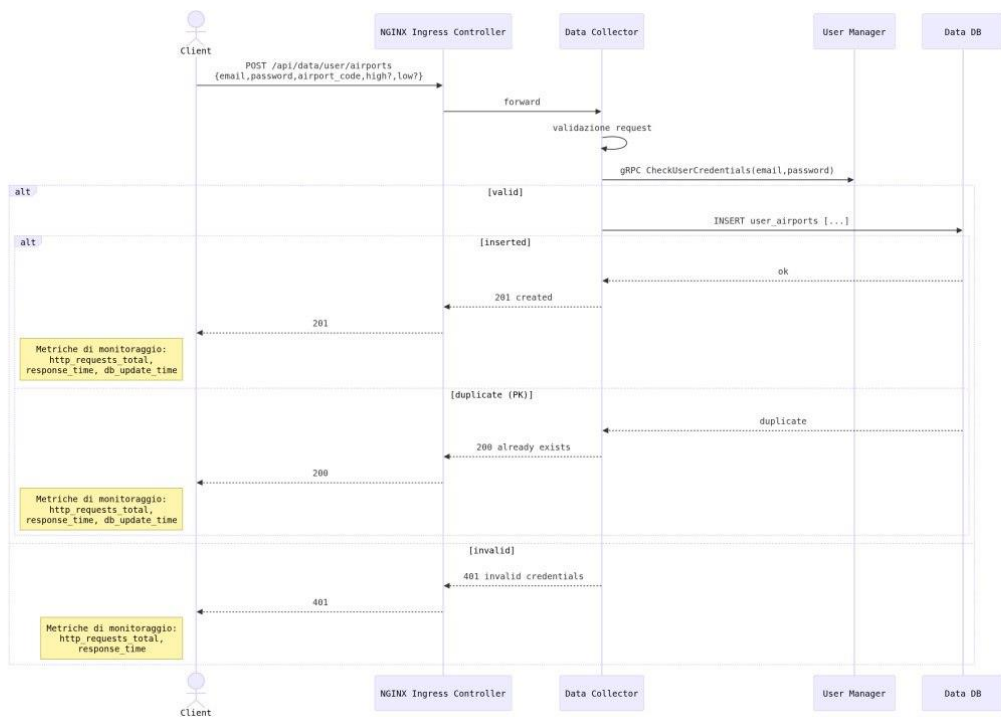


Figura 5: UML - Data Collector - Aggiunta aeroporto di interesse per un utente

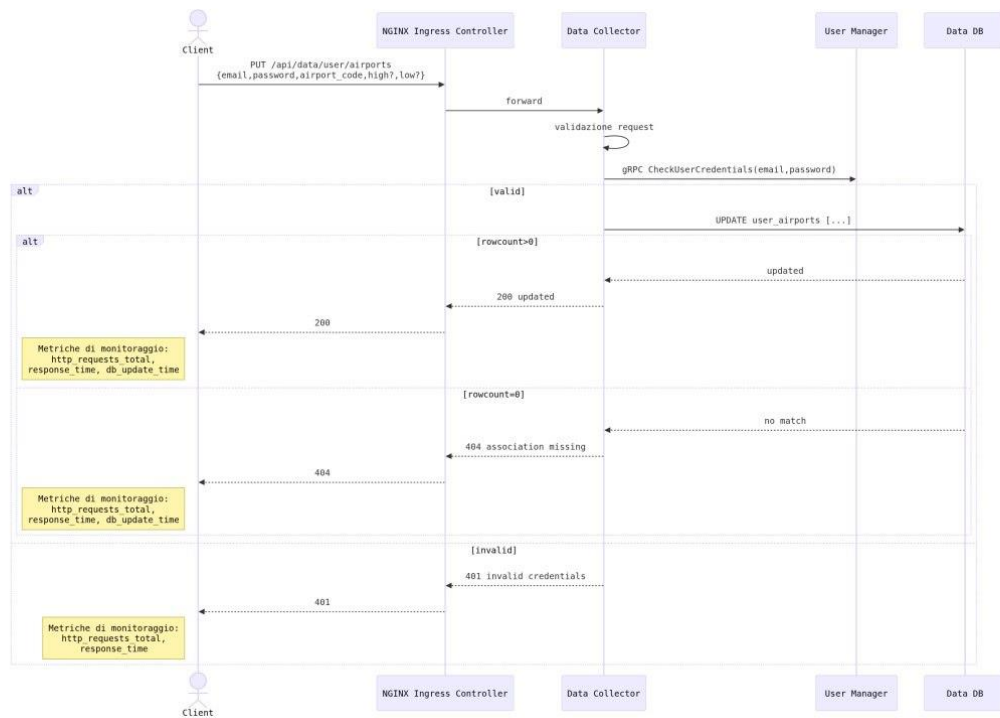


Figura 6: UML - Data Collector – Aggiornamento soglie high_value / low_value per un profilo utente-aeroporto

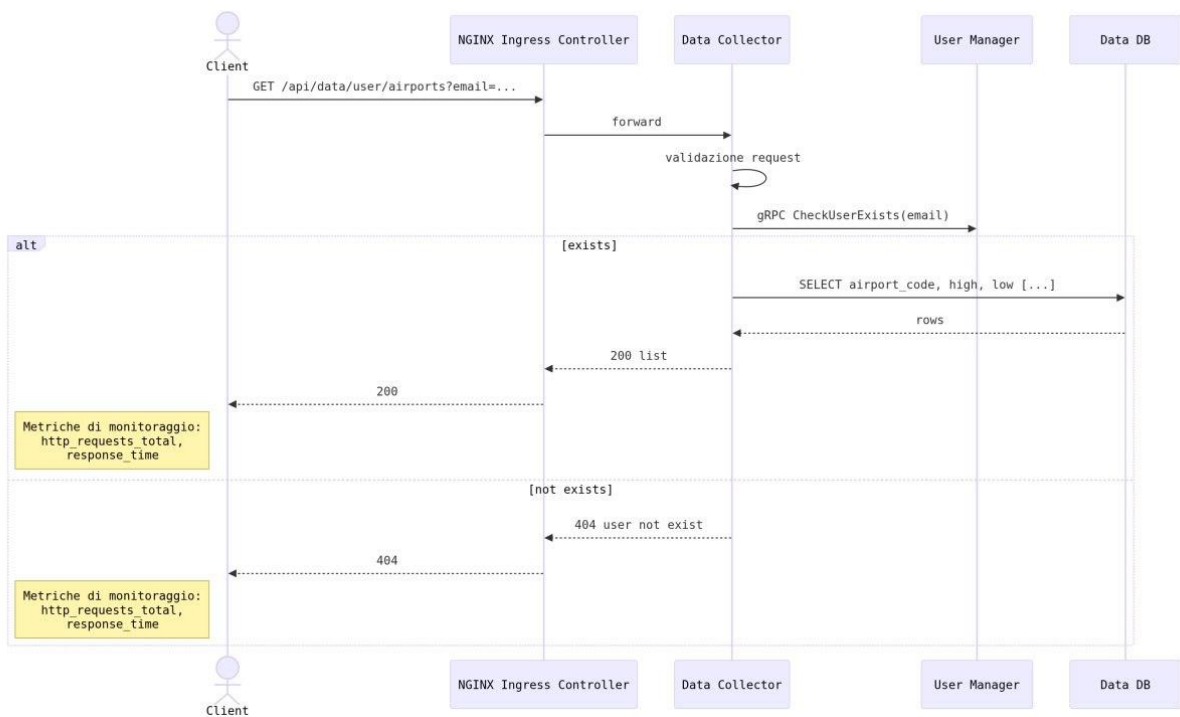


Figura 7: UML - Data Collector - Lista aeroporti di interesse per un utente

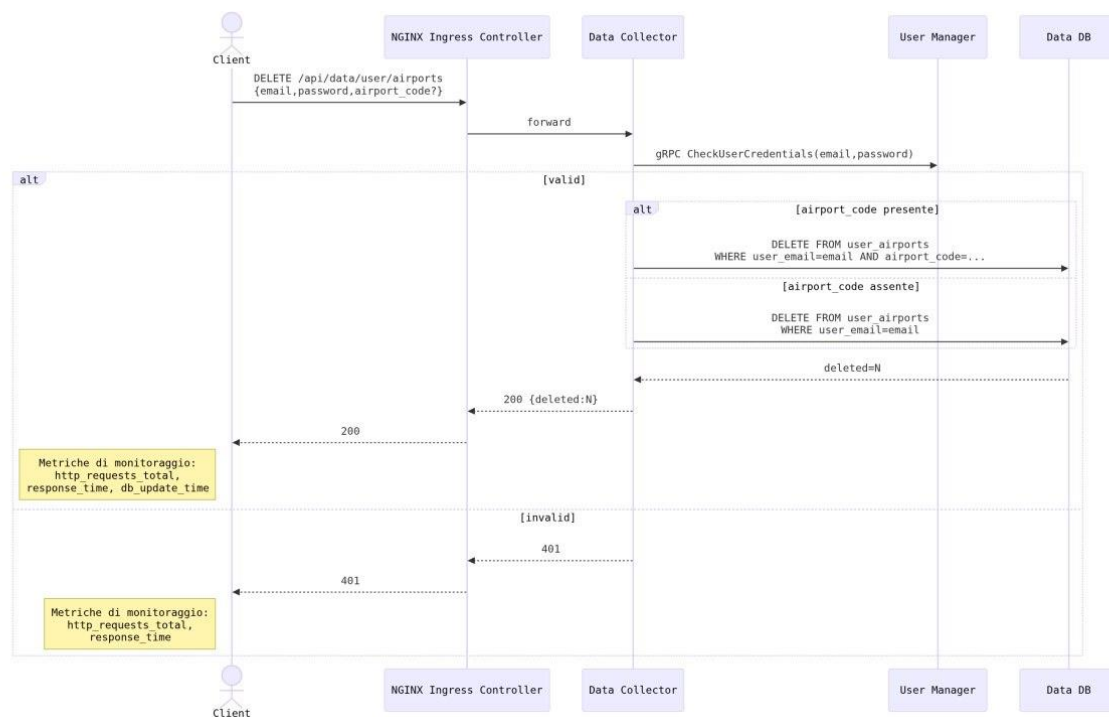


Figura 8: UML - Data Collector – Rimozione associazione utente-aeroporto

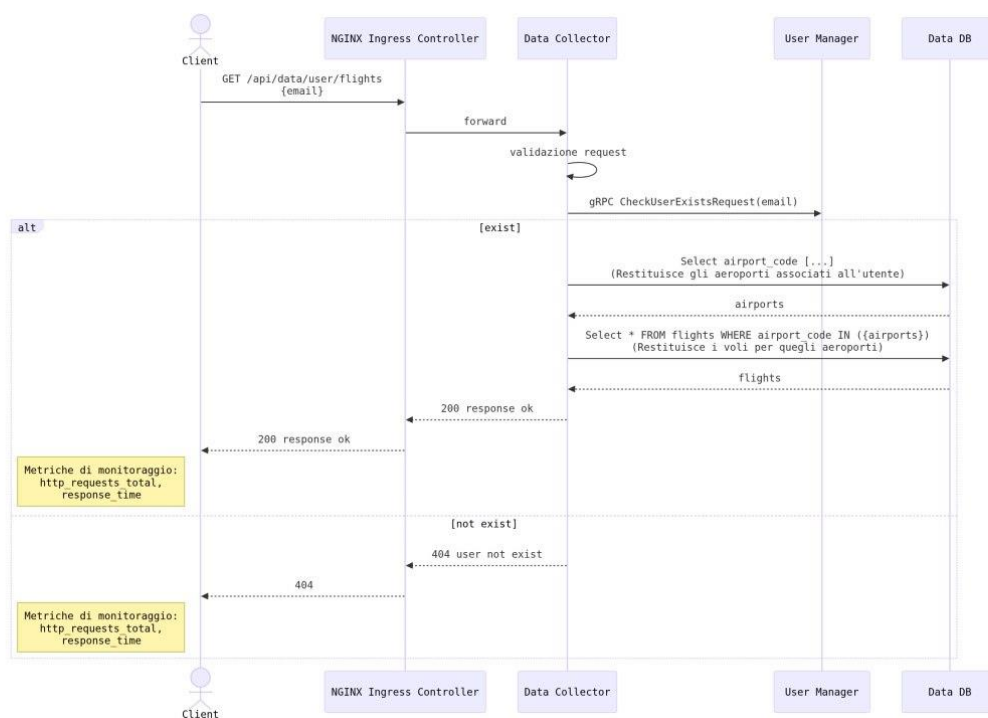


Figura 9: UML - Data Collector - Lista voli registrati nel DB data_db per gli aeroporti di interesse di un utente

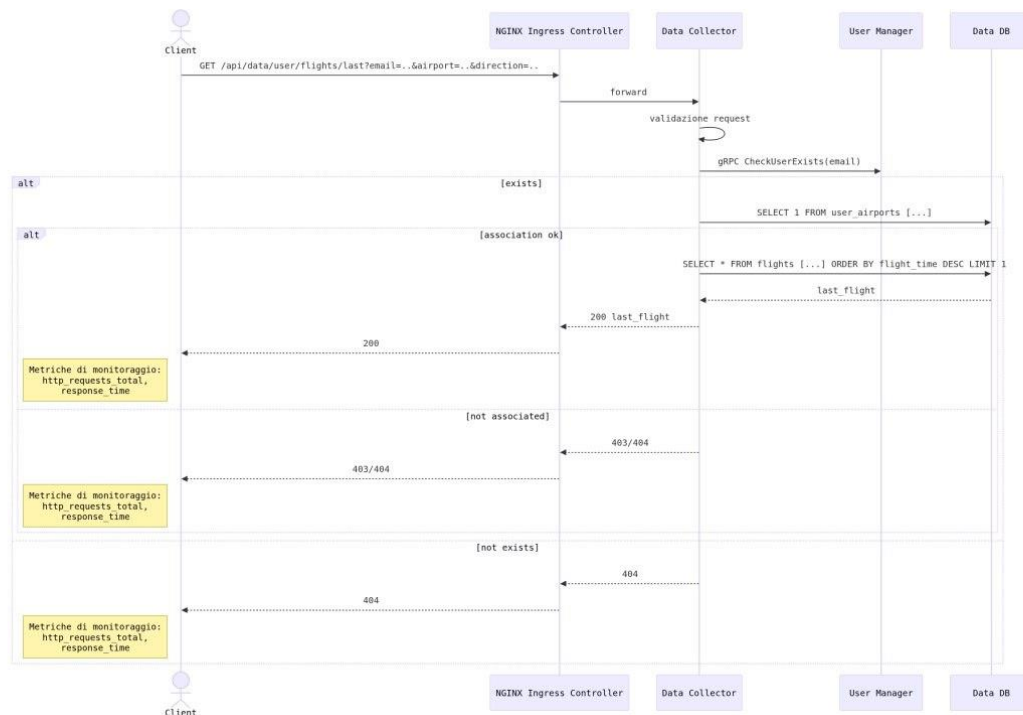


Figura 10: UML - Data Collector - Ultimi voli in arrivo e in partenza registrati nel DB data_db per un specifico aeroporto di interesse per l'utente

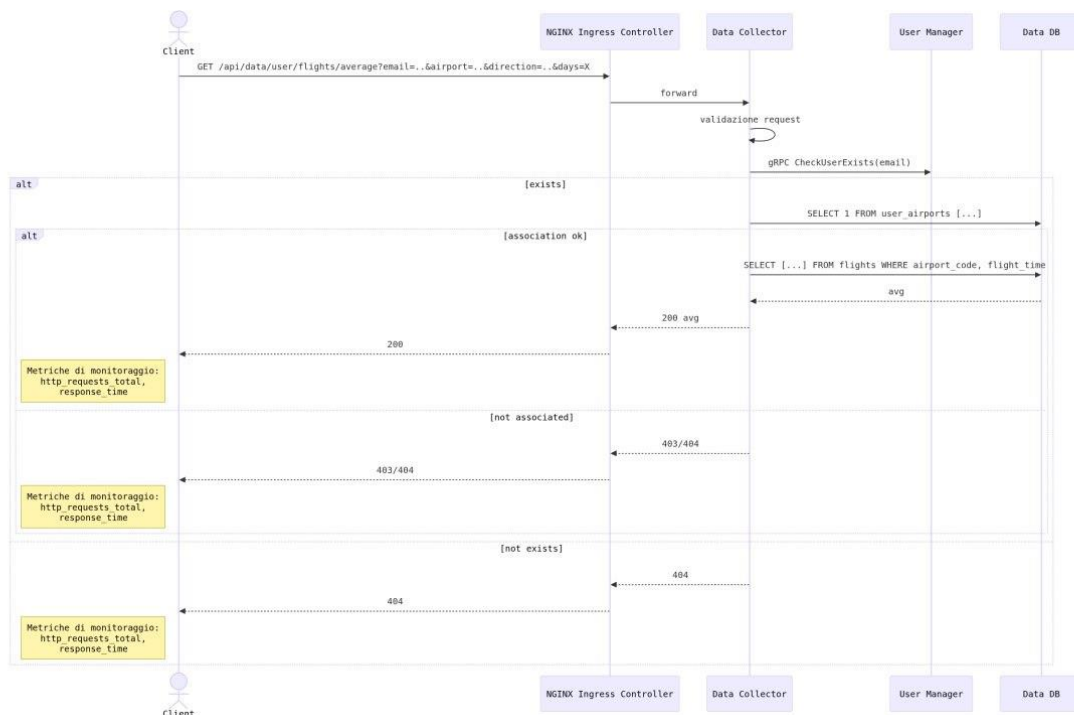


Figura 11: UML - Data Collector – Calcolo media di voli in partenza e in arrivo da un dato aeroporto negli ultimi x giorni

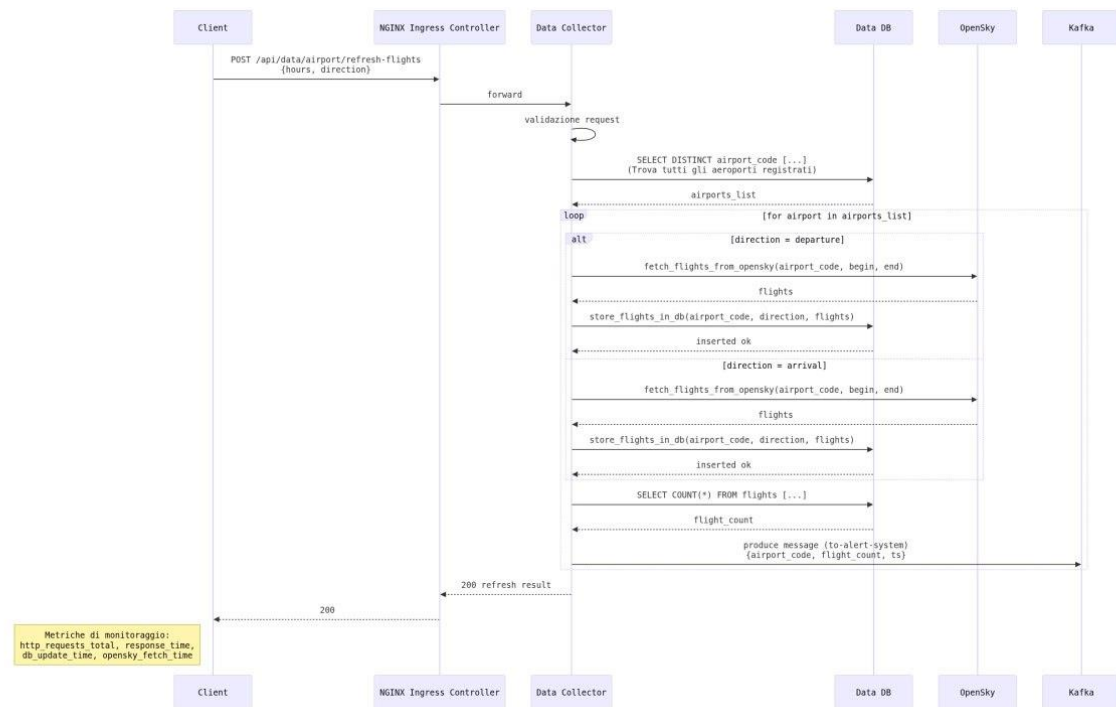


Figura 12: UML - Data Collector – Aggiornamento voli per tutti gli aeroporti di interesse

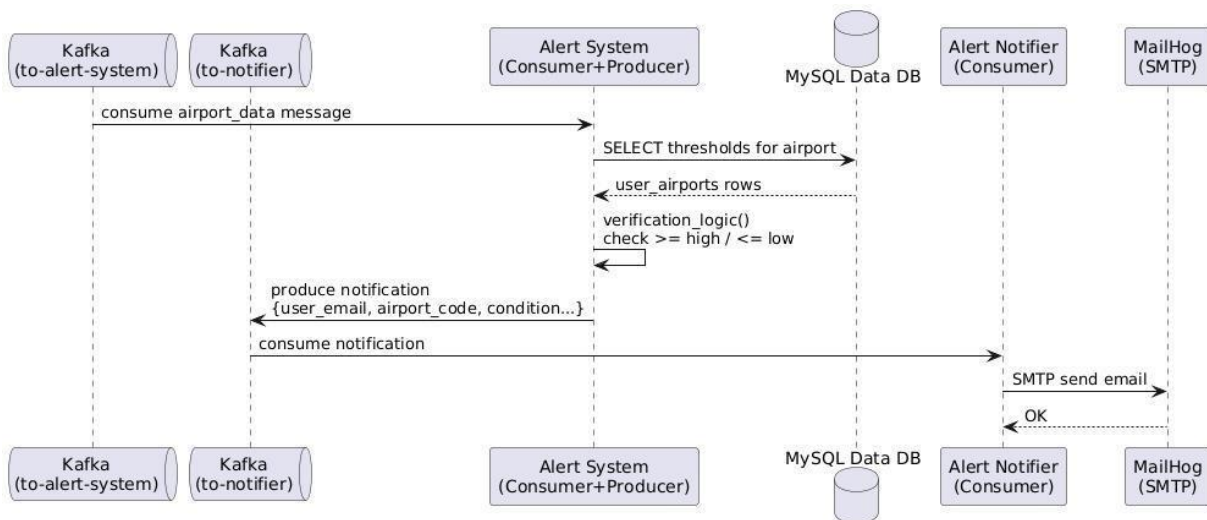


Figura 13: UML - Notifiche Kafka

V. Generalità sulla soluzione proposta e conclusioni

L'obiettivo principale di questo homework è stato quello di introdurre osservabilità, monitoraggio e orchestrazione

- **Introduzione dell'osservabilità applicativa tramite Prometheus**

L'integrazione di Prometheus ha consentito di arricchire l'architettura con un livello di osservabilità esplicito e strutturato. Ogni microservizio espone metriche applicative dedicate (numero di richieste, errori, tempi di risposta, tempi di accesso al database e tempi di fetch verso OpenSky), permettendo di monitorare il comportamento del sistema non solo a livello infrastrutturale ma anche logico-funzionale.

- **Preparazione all'ambiente Kubernetes**

È stato realizzato il passaggio da Docker Compose a Kubernetes. La definizione dell'ingress, dei manifesti dei microservizi, dei secret e delle risorse di rete consente di replicare l'architettura distribuita in un contesto orchestrato, mantenendo le stesse interazioni funzionali già validate. L'uso di Kind come cluster locale permette di testare l'intero cluster Kubernetes in modo riproducibile, senza dipendenze da infrastrutture esterne.

- **Adozione di Ingress e visione sistemica delle interazioni**

L'introduzione dell'Ingress Controller consente di centralizzare l'accesso all'applicazione, mantenendo trasparente la complessità interna del sistema. Le interazioni tra mondo esterno, gateway, microservizi, sistemi di messaggistica e monitoring sono state mantenute.

In conclusione, il lavoro svolto nei tre homework mostra una progressione coerente e incrementale, in cui ogni estensione si innesta naturalmente sulla precedente. L'adozione di Prometheus e Kubernetes non rappresenta un'aggiunta isolata, ma il completamento di un'architettura pensata fin dall'inizio per essere distribuita, monitorabile e pronta per scenari di utilizzo realistici.